



Java程序设计

Java异常处理

余 烨

本章学习提示

- 本章我们主要学习Java语言的异常处理机制，内容包括：
  - 什么是异常？有哪些异常？
  - 如何处理异常？
  - 定义自己的异常类

引 言

- 异常机制已经成为一门编程语言是否成熟的标准
  - C++、C#、Ruby、Python
- 对于一个程序设计人员，需要尽可能地预知所有可能发生的情况，尽可能保证程序在所有糟糕的情况下都能运行

人算不如天算！

- 增加了异常处理机制后的程序具有更好的容错性、更加健壮

引 例

- 在下五子棋时，判断用户的输入是否合法，在合法的情况下继续进行操作，否则，提示用户错误信息，要求用户重新输入。

```
if (用户输入包含除逗号之外的其他非数字字符 )
{ alert 坐标只能是数值；  goto retry}
else if ( 用户输入不包含逗号
{alert 应使用逗号分隔两个
else if (用户输入坐标值超出了有效范围
{alert 用户输入坐标应在棋盘坐标之内；  goto retry}
else if( 用户输入的坐标已有棋子 )
{alert 只能在没有棋子的地方下棋;
else                                  ●●●●●●
{  //业务实现代码 …… }
```

代码急剧增加

世界上的意外
是不可枚举的！

为什么要有异常处理机制

- 缺点
  - 无法穷举所有异常情况
  - 错误处理代码和业务实现代码混杂
  - 代码量增加



```
if ( 用户输入不合法 )
{ alert 输入不合法;    goto retry}
else
{   //业务实现代码 …… }
```



异常处理机制：当程序运行出现意外情形时，系统会自动生成一个Exception对象来通知程序

什么是异常

- 软件开发和运行时，可能会发生下面情况：
  - 想要打开的文件不存在
  - 网络连接中断
  - 操作数超出预定范围
  - 正在装载的类文件丢失
  - 访问的数据库打不开
  - ……

这些并不是因为程序的错误引起的！

- **异常** 指程序运行过程中出现的非正常现象，例如用户输入错误、除数为零、需要处理的文件不存在、数组下标越界等。

程序无法控制的外部环境问题！

异常处理

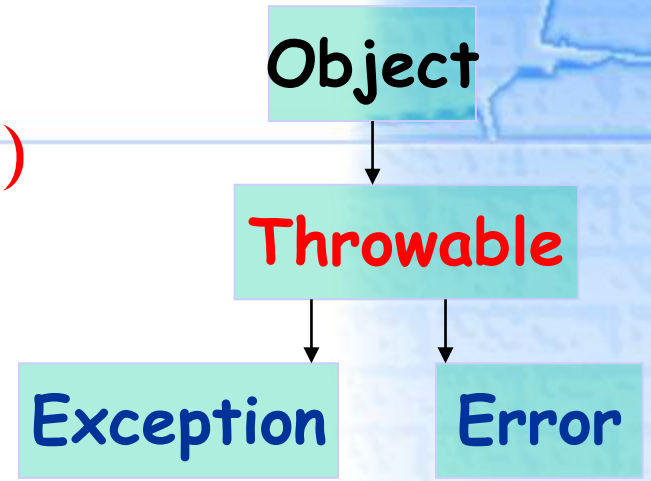
- 由于异常情况总是难免的，良好的应用程序除了具备用户所要求的功能以外，还应该具备预见并处理可能发生的各种异常的功能。这种对异常情况进行处理的技术成为异常处理。
- JAVA提供了功能强大的异常处理机制，可以方便地
  - 在程序中监视可能发生异常的程序块
  - 并将所有异常处理代码集中放置在程序的某处

  使完成正常功能的程序代码与进行异常处理的程序代码分开。
- Java中用2种方法处理异常：
  - 在发生异常的地方直接处理
  - 将异常抛给调用者，让调用者处理

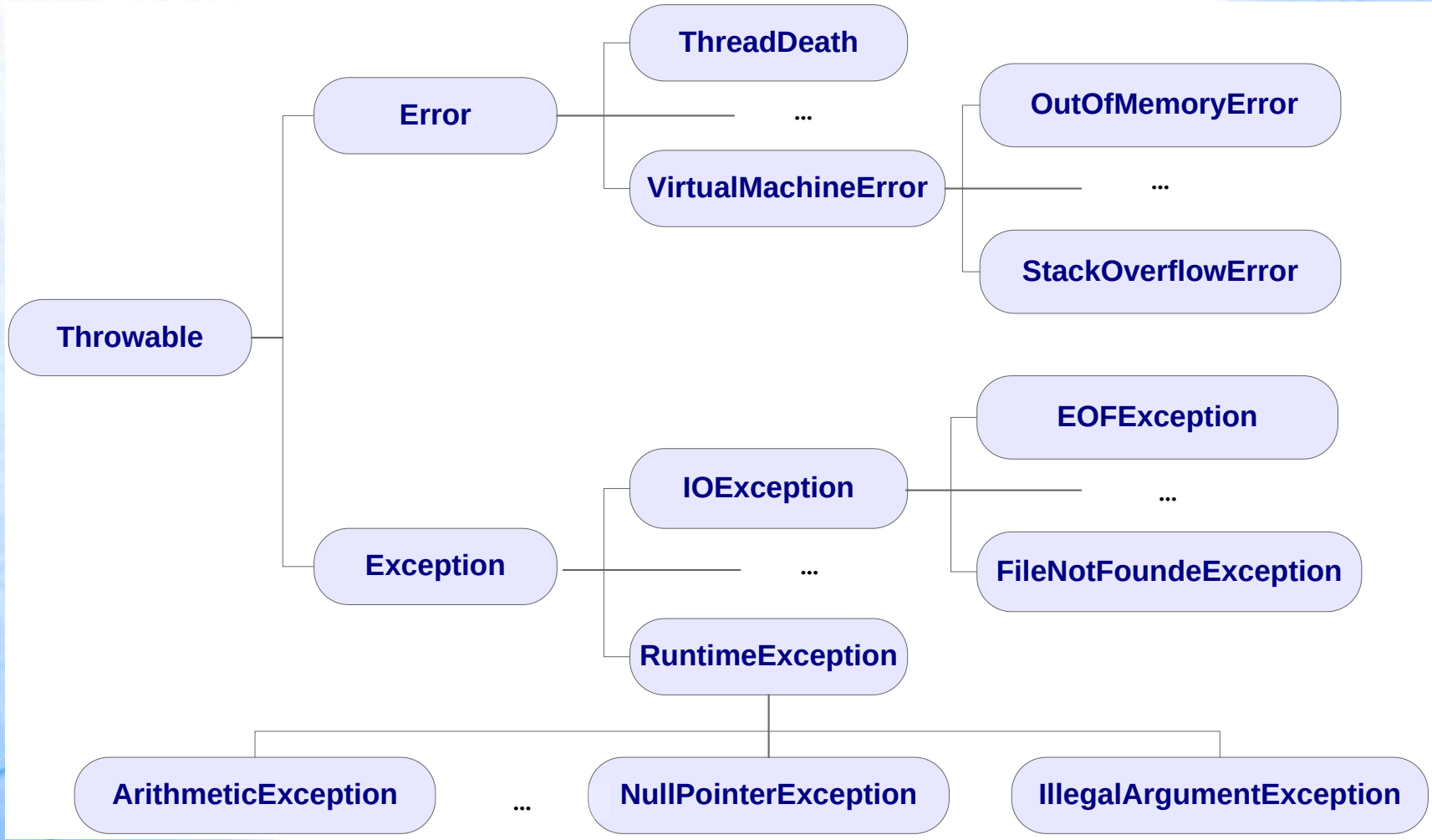
异常类

- 在JAVA的异常处理机制种引进了很多用来描述和处理异常的类，称为**异常类**。每个异常类反映一类**运行错误**，类定义中包含了该类异常的信息和对异常进行处理的方法
  - 程序执行中如出现异常，系统会检测到并自动生成一个相应的异常类对象，然后交给运行时系统
  - 运行时系统自动寻找相应的异常处理代码处理这一异常。若找不到可以处理该异常的代码，则运行时系统将终止，程序运行将退出。

异常分类



- JAVA将异常分为Exception（异常）和Error（错误）两大类
  - 同：均位于java.lang包中；
    顶层是java.lang.Throwable类
  - 异：凡是Throwable的实例或者其子类的实例都可以被java虚拟机抛出
    - **Exception类**解决由程序本身及环境所产生的异常；**Error类**处理内部系统错误（Error指与虚拟机相关的问题）。
    - Exception类异常可以被捕获并进行处理，而对Error类异常，程序员通常无能为力，只能在其发生时由用户按照系统提示关闭程序（这种错误**无法恢复**或**不可能捕获**，将导致应用程序中断；这一类错误由Java运行时系统处理，不需要我们去处理）。
    - Exception类定义了程序遇到的**轻微**的错误条件；Error类定义了程序中不能恢复的**严重**错误条件。


Throwable
Error
ThreadDeath
...
VirtualMachineError
OutOfMemoryError
...
StackOverflowError
Exception
IOException
EOFException
...
FileNotFoundeException
...
RuntimeException
ArithmeticException
...
NullPointerException
IllegalArgumentException

常用的异常类

- Exception
  - InterruptedException
  - ClassNotFoundException
  - ClassNotSupporteException
  - IllegalAccessException
  - InstantiationException
  - NoSuchMethodException
  - RuntimeException
    - ArithmeticException
    - ArrayStoreException
    - ClassCastException
    - IllegalArgumenException
    - IllegalThreadStateException
    - NumberFormatException
    - ArrayIndexOutOfBoundsException
    - StringIndexOutOfBoundsException

访问异常信息

- public Exception(String message);

  ……其它构造方法……

  message: 异常的详细描述

  ArithmeticException

- public String **getMessage()**; **/ by zero**

  Returns the detail message string of the Exception.

- public String **toString()**; **java.lang.ArithmeticException: / by zero**

  Returns a short description of the Exception.

- public void **printStackTrace()**;

  Prints the Exception and its backtrace to the standard error stream.

java.lang.ArithmeticException: / by zero
at ExceptionTest1.main(ExceptionTest1.java:13)

示例

```
try {
      FileInputStream fis = new FileInputStream("a.txt");   }
catch(IOException ioe)  {
      System.out.println(ioe.getMessage());
      System.out.println(ioe.toString());
      ioe.printStackTrace();
}
```

```
--------------------Configuration: <Default>--------------------
a.txt (系统找不到指定的文件。)
java.io.FileNotFoundException: a.txt (系统找不到指定的文件。)
java.io.FileNotFoundException: a.txt (系统找不到指定的文件。)
        at java.io.FileInputStream.open(Native Method)
        at java.io.FileInputStream.<init>(FileInputStream.java:138)
        at java.io.FileInputStream.<init>(FileInputStream.java:97)
        at AccessExceptionMsg.main(AccessExceptionMsg.java:10)

Process completed.
```

AccessExceptionMs.java

异常处理方法

- try…catch…
- finally
- throws
- throw

try…catch

- 语法：

```
try{
    //业务实现代码（可能会发生异常的语句）}
catch(异常对象){
    //
```

A catch block is often referred to as an exception handler

- 程序执行过程中，如果没有出现异常，将正常执行，catch块不起作用，若出现异常将终止try块代码的执行，自动跳转到所发生的异常类对应的代码块中，执行该块中的代码。
- 如果执行try块里的业务逻辑代码出现异常，系统自动生成一个异常对象，该异常对象被提交给java运行时环境，这个过程称为抛出异常
- 当java运行时环境收到异常对象时，就会寻找能处理该异常对象的catch块，如果找到合适的catch块并把该异常对象交给catch块处理，这个过程称为捕获异常

```
//五子棋之例
String inputStr = null;
BufferedReader br = new BufferedReader(new InputStreamReader( System.in) );
while ( (inputStr = br.readLine()) != null )  {
    try {
        String[] posStrArr = inputStr.split(",");
        int xPos = ...
        ...
        if (...)
            System.out.println(...有棋子，重新输入");
        }
        grid[xPos-1][yPos-1] = '@';  }
    catch(Exception e)  {
        System.out.println("您输入的坐标不合法，请重新输入，下棋坐标应为x，y的格式");
    }  }
```

字母不能正确解析
没有逗号不能正确解析
解析出来的坐标引起数组越界

ChessTest.java

try…catch

- 执行顺序
  - 阅读下面代码，分析若发生异常，会输出什么？没有发生异常，会输出什么？

```
try{
        FileReader fr = new FileReader("aa.text");
        System.out.println(“go on”);
}
catch(Exception e)
{
        e.getMessage();
        //e.printStackTrace();
        System.out.println(“Catched”);
}
System.out.println(“Completed”);
```

程序在抛出异常的地方就终止执行

最大、最小异常

FileNotFoundException e

ExceptionTest2.java

多个异常处理

```
int x = (int) (Math.random() * 5);
int y = (int) (Math.random() * 10);
int[] z = new int[5];
try
{
    System.out.println("y/x=" + (y/x));
    System.out.println("y=" + y + "z[y]=" + z[y];
}
catch (ArithmeticException exc1)
{   System.out.println("算术运算异常：" + exc1.getMessage()); }
catch (ArrayIndexOutOfBoundsException exc2)
{   System.out.println("数据越界异常：" + exc2.getMessage()); }
```

```
catch (Exception e)
{   System.out.println(e.toString()); }
```

处理规则：越特殊的异常越在前面处理，越普遍的异常越在后面处理

ExceptionTest3.java

finally语句

- 如果把finally块置try…catch….语句后，finally块一般都会得到执行
- 一般把需要关闭的物理资源（如：数据库连接、网络连接、磁盘文件等...）等的关闭放在finally语句中
- try…finally; try…catch; try…catch…finally 三种形式
- 以下特殊情况下，finally块不会被执行：
  - finally块中发生了异常
  - 程序所在线程死亡
  - 在前面代码中用了System.exit();
  - 关闭了CPU

示 例

```
FileReader fr;
try{
        fr = new FileReader("aa.txt");
}
catch(Exception e) {
        e.printStackTrace();
        System.exit(1);
}
finally{
        if (fr!=null)
        {  try{
                fr.close();
           } catch{Exception e)
           {  e.printStackTrace(); }
        }
}
```

ExceptionTest3.java

Example1

```
//捕获数组下界越界异常
public class ExceptionDemo01 {
    public static void main(String args[ ])  {
        try {
              int a[ ]={1,2,3},sum=0;
              for(int i=0; i<=3; i++)  sum += a[i];
              System.out.println("sum=" + sum + "Successfully!");
        }
        catch ( ArrayIndexOutOfBoundsException e ) {

     System.out.println("ArrayIndexOutOfBoundsException
     detected");
               // e.printStackTrace();
              System.out.println(“ 数组内仅3元素!");
        }
          finally {
               System.out.println("Program Finished !"); }
    } }
```

Test_Excption.java

Example 2

```
//使用FileInputStream从文件中读取字节流
import java.io.*;
public class TestFileInputStream_Exception {
  public static void main(String[] args) {
    int b = 0;     long num = 0;
    FileInputStream in = null;
    try {
        in = new FileInputStream("C:/io/TestFileInputStream.java");
        while((b=in.read())!=-1){//一次读一个字节,判断是否读到文件结尾
            System.out.print((char)b);
            num++; }
        in.close();
        System.out.println("\n共读取了 "+num+" 个字节");
    } catch (FileNotFoundException e) {
        System.out.println("找不到指定文件");
        System.exit(-1);
    } catch (IOException e1) {
      System.out.println("文件读取错误");
      System.exit(-1);
    } } }
```

TestFileInputStream_Excption.java

Tip:Exception Controlled Loops

- 有时处理从键盘等终端接收的一个输入，比如说要求输入整数，但如果输入非整数的话，会发生异常，但程序不能因为这个异常而退出，因此，需要进行一个处理。这里建议用Exception Controlled loop来处理
- InputMismatchException（位于java.util包）

```
boolean done = false;
while (!done)
{
    try
    {
        引起异常的代码.
        done = true; //结束循环.
        <更多代码>
    }
    catch(Exception_Class e)
    {
        <代码>
    }
}
```

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class InputMismatchExceptionDemo {
    public static void main(String[] args) {
        Scanner keyboard = new Scanner(System.in);
        int number = 0;
        boolean done = false;
        while (! done) {
            try {
                System.out.println("Enter a whole number:");
                number = keyboard.nextInt();
                done = true;  }
            catch(InputMismatchException e) {
                keyboard.nextLine();
                System.out.println("Not a correctly written whole number.");
                System.out.println("Try again.");  }  } //end of while
} }
```

ExceptionTest4.java

声明抛出异常throws

- 不进行任何捕获，抛出异常：Throws

```
import java.io.*;
public class Test3 {
        public static void main(String[] args) Throws Exception {
                Father father = new Father();
                Father.test1();    }  }
class Father {
        private Son son = null;
        public Father()  {
                son = new Son();  }
        public void test1() Throws Exception{
                System.out.println("1");
                son.test2();    }  }
 class Son {
        public void test2() throws Exception {
                FileReader fr = null;
                fr = new FileReader("test.txt");
        } }
```

```
try{
      son.test2();}
catch(Exception e {

   e.printStackTrace();
```

不能是FileNotFoundException，因为抛出的是Exception

ExceptionTest5.java

子类中的throws语句

如果子类中重写了父类中的方法，而父类中的方法有抛出异常的话，则子类中也需要抛出异常，子类中抛出的异常可以和父类中抛出的异常相同，也可以是父类中抛出异常的子集或子类，但，不能抛出新的异常。（构造方法除外）

```
class Father {
        public void test1()  throws Exception{
                ***********
                System.out.println("1");
        }  }
 class Son extends Father{
        public void test1() throws FileNotFoundException {
                FileReader fr = null;
                fr = new FileReader("test.txt");
        }  }
```

ExceptionTest6.java

抛出异常throw

- 在程序中自行抛出异常

Every exception has a String instance variable that contains some message, which typically identifies the reason for the exception

```
try {
    if (men == 0 && women == 0)
        throw new Exception("Lesson is canceled. No students.");
    else if (men == 0)
        throw new Exception("Lesson is canceled. No men.");
    else if (women == 0)
        throw new Exception("Lesson is canceled. No women.");
    if (women >= men)
        System.out.println("Each man must dance with " + women/(double)men + "
        women.");
    else
        System.out.println("Each woman must dance with " + men/( double)women
        + " men.");
}
catch (Exception e) {
    System.out.println(e.getMessage());
    System.exit(0);
}
```

Returns the string

Try…throw…catch 机制

ExceptionTest7.java

```
public class Example7_4
{
  public static void main(String args[ ])
   {
     int n = 0, m = 0, t = 1000;
     try
     {
          m = Integer.parseInt("8888");
          n = Integer.parseInt("ab89"); //发生异常,转向catch
          t = 7777;  //t没有机会被赋值
     }
     catch(NumberFormatException e)
     {
          System.out.println("发生异常:"+e.getMessage());
     }

     System.out.println("n="+n+", m="+m+", t="+t);

     try
     {
          System.out.println("故意抛出I/O异常！ ");
          throw new java.io.IOException("我是故意的");
          //System.out.println("这个输出语句肯定没有机会执行,所以必须注释掉，否则编译出错");
     }
     catch(java.io.IOException e) {
          System.out.println("发生异常:"+e.getMessage());
     }
  }
}
```

异常分类

- Checked Exception（检查性异常）
  - 程序代码还没运行，在编译的时候就能够判断可能出现的异常，要求编程者必须处理，又称编译异常
  - 所谓检查（Checked）是指编译器要检查这类异常，检查的目的一方面是因为该类异常的发生难以避免，另一方面就是让开发者去解决掉这类异常，所以称为必须处理（try ...catch）的异常。如果不处理这类异常，集成开发环境中的编译器一般会给出错误提示。
- UnChecked Exception（非检查性异常）
  - 在编译时检查不出来，在运行时才能发现，又称运行时异常
  - 非检查（Unchecked）是指编译器不会检查这类异常，不检查的则开发者在代码的编辑编译阶段就不是必须处理，这类异常一般可以避免，因此无需处理（try ...catch）。如果不处理这类异常，集成开发环境中的编译器也不会给出错误提示。

Checked Exception

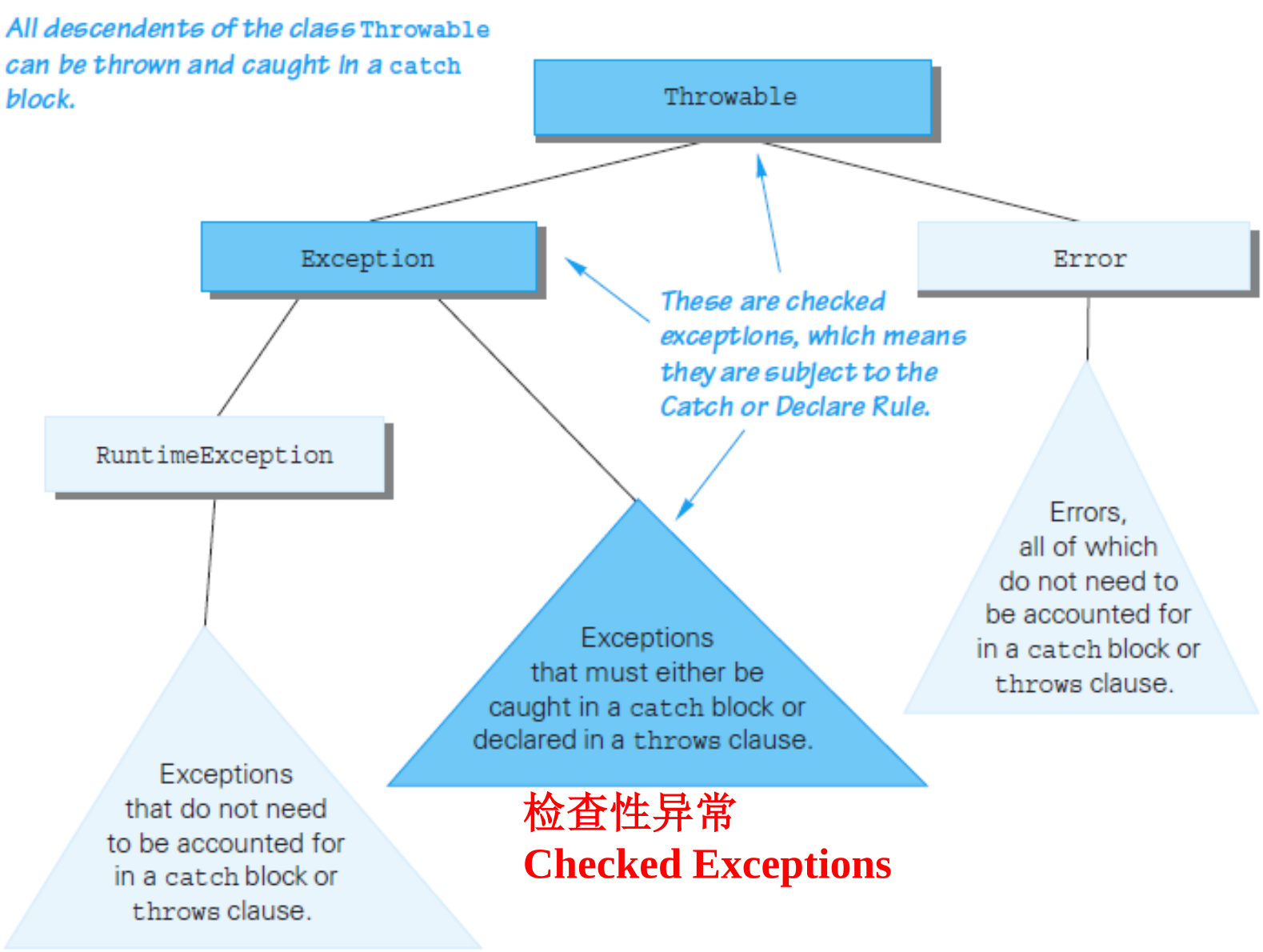
- 例如：一个读取文件的方法代码逻辑没有错误，但程序运行时可能会因为文件找不到而抛出FileNotFoundException，如果不处理这些异常，程序将来肯定会出错。所以编译器会提示你要去捕获并处理这种可能发生的异常，不处理就不能通过编译。



Unchecked Exception

- 例如：你的程序逻辑本身有问题，比如数组越界、访问null对象，这种错误你自己是可以避免的。编译器不会强制你检查这种异常。





检查性异常

Checked Exceptions

Catch or Declare Rule

- 捕获异常或者声明抛出异常
  - 如果throw语句抛出的异常是Checked异常，那么该语句要么处于try块里，显式捕获该异常，要么放在一个带throws声明抛出的方法中，即把该异常交给该方法的调用者处理
  - 如果抛出的是Runtime异常，则无需如此

```
public static double safeDivide( int top, int bottom) throws
DivisionByZeroException
{
    if (bottom == 0)
    throw new DivisionByZeroException();
    return top/( double)bottom; }
```

自定义异常类

- 自定义异常类可以是其他已存在异常的子类
- 定义异常类时，通常构造函数的定义最重要；可以不定义其他的方法（从父类中继承而来）

```
public class DivisionByZeroException extends Exception
{
        public DivisionByZeroException()
        {   super ("Division by Zero!");  }
        public DivisionByZeroException(String message)
        {    super(message);    }
}
```

getMessage()方法
发送的消息非常重要

```
Throw new DivisionByZeroException(“Oops! ”);
System.out.println(e.getMessage());
```

```
class OverFlowException extends Exception {//自定义异常类
    OverFlowException()   {
        System.out.println("此处数据有溢出，溢出类是OverFlowException");}
}
public class ExceptionDemo04 {
   public static int x=100000;
   public static int multi() throws OverFlowException    {
      int aim;      aim=x*x*x;
      if(aim>2.15E9||aim<0) {   throw new OverFlowException(); }
      else
          return x*x;  }
   public static void main(String args[]){
      int y;
      try{    y=multi();     System.out.println(" y="+y);
      }
      catch( OverFlowException e){//捕获异常并处理
        System.out.println(e); } } }
```

改成e.getMessage()
输出null

ExceptionTest8.java

此处数据有溢出，溢出类是OverFlowException
OverFlowException

异常处理规则

- 不要过度使用异常
  - 把异常和错误混淆在一起，不编写任何错误处理代码，而是简单地抛出异常来代替所有错误处理
  - 使用异常处理来代替流程控制
- 不要使用过于庞大的try块
  - try块过于庞大，难免后面会跟大量的catch块，此时需要分析他们之间的逻辑关系，反而导致了编程复杂度的增加
- 尽量提供对每个最小异常的处理
  - 若每个异常都采用相同的处理方式，会导致无法对不通过异常分情况处理，以跟踪错误发生的原因
- 不要忽略捕获到的异常
  - 在合适的层处理异常

回顾

- 什么是异常？Java中有哪些常用的异常类
- try…catch…finally
- throws、throw（try…throw…catch）语句
- 自定义异常类

Class is dimissd!

